RAG End-to-End (from scratch)

Modulo 3 · Lezione 2



La lezione scorsa: l'anatomia e il catalogo

INGEST
`indicizza_corpus()`
→
RETRIEVE
`cerca_chunk()`
→
GENERATE
`genera_risposta()`

Il progetto del Modulo 2 ha un nome — **RAG** — e un'anatomia. E ha un catalogo di failure, compilato insieme.

`IL CATALOGO`

retrieval miss
context overflow
hallucination nonostante il contesto
lost-in-the-middle

Stasera si costruisce **la pipeline unica del modulo** — configurabile e ispezionabile: la base su cui i confronti delle prossime lezioni hanno senso.



Agenda

	Blocco	Dove	Tempo
	Da script sparso a componente	slide	30′
	Live coding: `indicizza()`	`working/rag.py`	30′
	Esercizio: `recupera()` e `rispondi()`	`working/rag.py`	45′
	Reveal e ponte	slide	15′

Un solo file per tutta la serata: `working/rag.py`.



app.py com’è rimasto

```
# il Progetto Modulo 2, quattro funzioni in fila
MODELLO = "gpt-4o-mini"
CHUNK_SIZE = 500

def indicizza_corpus(): …
def cerca_chunk(query, k=4): …
def genera_risposta(query, chunks): …
def estrai_fattura(path): …
```

«Cosa succede se porto k a 8?»

Per rispondere bisogna sapere **dove** mettere le mani: il modello sta in una funzione, il k in un default, il chunk size a metà file.

Funziona. Ma nel Modulo 3 passeremo il tempo a **girare manopole**: la prima cosa da fare è metterle sullo stesso pannello.



Il salto: da script a componente

1 · UN POSTO SOLO

Tutti i parametri in un **punto unico di configurazione**: il dizionario `CONFIG` in cima al file.

2 · RISPOSTA + CHUNK

La risposta torna **insieme ai chunk usati**: `rispondi()` → `(risposta, chunk)`.

Il salto rispetto ad `app.py` è tutto qui. Il resto — load, clean, chunk, embed, il prompt con le citazioni — è già stato scritto nel Modulo 2: si importa da `aikit/` e si riusa.



Le tre interfacce della pipeline

`indicizza()`	l'**INGEST**: corpus → chunk → embedding → collection	`offline · una tantum`
`recupera(query, k)`	il **RETRIEVE**: i k chunk più pertinenti, con id, testo e score	`online · a ogni domanda`
`rispondi(query)`	il **GENERATE**: risposta con citazioni, insieme ai chunk usati	`online · a ogni domanda`

> L'anatomia della lezione scorsa, diventata **contratto di codice**: tre firme, un file.



La risposta torna con i chunk

MODULO 2 · app.py

`genera_risposta() → risposta`

Se cita male, chi è il colpevole? Il retrieval o il prompt? **Non si può dire.**

DA STASERA · rag.py

`rispondi() → (risposta, chunk)`

`chunk 1` `chunk 2` `chunk 3` `chunk 4`

Cosa è entrato nel prompt si **ispeziona**, sempre.

La lezione scorsa i chunk si aprivano a mano per diagnosticare i failure. Da stasera è cablato nella firma: è **lo strumento di debug del modulo**, non un dettaglio implementativo.



CONFIG: il punto unico di configurazione

```
CONFIG = {
    "corpus_dir": SCRIPTS / "dataset" / "lumen_m3",  # i sei documenti
    "collection": "lumen_m3",  # si indicizza una volta
    "chunk_size": 500,  # recursive, come nel Modulo 2
    "k": 4,  # chunk per domanda
    "modello": "gpt-4o-mini",  # il generator
}
```

Il criterio: qui dentro sta tutto ciò che nel modulo vorremo **cambiare per vederne l'effetto** — e nient'altro. Da Modulo 3 · Lezione 4 in poi, le manopole si girano da qui.



Il prompt RAG si riusa, non si riscrive

Quello di Modulo 2 · Lezione 11, dentro `rispondi()`

```
Rispondi usando SOLO le informazioni nei documenti.
Cita la fonte tra parentesi quadre. Se la risposta
non è nei documenti, rispondi: "Non lo trovo nei
documenti." — senza inventare nulla.

<documenti>
  …i chunk, ognuno con la sua fonte…
</documenti>

<domanda>
  …la domanda…
</domanda>
```

- risposta **solo dai documenti**
- citazione della fonte **tra parentesi quadre**
- **rifiuto pulito** quando la risposta non c'è



La citazione si verifica sui chunk

```
D: Quanto tempo ho per richiedere il contributo per la postazione di casa?

  1. guida-smart-working-004  score=0.581  La richiesta del contributo va presentata entro 60 giorni…
  2. guida-smart-working-003  score=0.418  Lavorare bene da casa richiede una postazione vera…

→ Hai 60 giorni dall'attivazione dello smart working per presentare la richiesta […] [guida-smart-working-004]
```

la fonte citata… → …è tra i chunk stampati… → …e contiene davvero i 60 giorni

Non più un atto di fede: **grounding verificabile**, chunk alla mano.



Il corpus del modulo: sei documenti Lumen

DAL MODULO 2

Manuale LumaDesk

il documento che conoscete meglio

DAL MODULO 2

Policy resi e garanzia

identica, byte per byte

DAL MODULO 2

Guida smart working

identica, byte per byte

NUOVO

Listino e catalogo

codici e sigle: LMX-4400, SRV-02, RMA-2

NUOVO

Ticket di supporto

schede brevi e gergali, casi che si somigliano

NUOVO

Changelog dei rilasci

versioni e numeri in mezzo ai paragrafi

Più realtà aziendale: **codici, gergo, versioni** — il materiale su cui il modulo lavorerà da qui in poi.



Un corpus uguale per tutti

MODULO 3

Lezione 2 · lo stesso corpus, gli stessi 82 chunk · Lezione 14 · Lezione 15: il tuo corpus

Nasce stasera ed è **già nella cartella** (`dataset/lumen_m3/`), generato da `make_dataset.py`: deterministico, stessi id e stessi chunk per tutti. Nessun comando da lanciare.

È la condizione per **confrontare**: tra due configurazioni del tuo RAG, e tra studenti al debrief.

Il corpus personale torna al Progetto (Modulo 3 · Lezione 15).



Live coding: indicizza()

REPLICATE — scrive il docente, voi con lui

`working/rag.py · TODO 1 · 30 minuti`

si rientra dopo il rilancio, chunk contati

`load` → `clean` → `chunk` → `embed` → `upsert in lumen_m3`

La filiera del Modulo 2 (Lezioni 4, 5, 10, 7, 9): la novità è che ogni passo legge i parametri da `CONFIG`.

Il file ha già CONFIG, firme e main: si riempie il corpo di `indicizza()`.



Cosa si è visto: 82 chunk, un rilancio senza doppioni

```
$ python working/rag.py
  changelog-rilasci: 11 chunk
  guida-smart-working: 13 chunk
  listino-catalogo: 11 chunk
  manuale-lumadesk: 16 chunk
  policy-resi-garanzia: 16 chunk
  ticket-supporto: 15 chunk
  [openai] 82 testi, 8547 token → $0.000171
indicizzati 82 chunk da 6 documenti in 'lumen_m3'
```

2° LANCIO

Ancora 82 chunk: il main re-indicizza solo a collection vuota — e richiamando `indicizza()`, `crea_collection` ricrea da zero. Mai doppioni.

COSTO

L'ingest dell'intero corpus costa **meno di un centesimo**: re-indicizzare è un problema di disciplina, non di soldi.

La collection vivrà tredici lezioni: "com'è fatta adesso" deve avere sempre una risposta certa.



Esercizio: recupera() e rispondi()

SCRIVETE VOI — da soli, con affiancamento

`working/rag.py · TODO 2-3 · 45 minuti · consegna in CONSEGNA.md`

si rientra per il reveal

`TODO 2 · recupera(query, k)`

i k chunk più pertinenti (k da CONFIG), con id, testo e score

`TODO 3 · rispondi(query)`

il giro completo col prompt di Modulo 2 · Lezione 11 → (risposta, chunk)

`FATTO QUANDO`

Domanda nel corpus → la risposta cita una fonte che sta tra i chunk stampati e la contiene davvero. Domanda fuori corpus → rifiuto pulito, niente inventato. Chi finisce: gira il k in CONFIG (2, poi 8) e guarda cosa cambia, poi domande tue sui documenti nuovi.



Reveal: due rispondi() a confronto

```
# citazione in coda
→ Hai 60 giorni dall'attivazione per presentare la
richiesta. [guida-smart-working-004]
```

```
# fonti in elenco
→ La finestra è di 60 giorni dall'attivazione.
  Fonti: guida-smart-working-004
```

> Formati diversi, stesso contratto: risposta + chunk usati, citazione che punta a un chunk **vero**.



La stessa pipeline su un follow-up

```
D: Perché il piano si ferma sempre a metà corsa?
→ …una memoria di calibrazione corrotta, caso tipico dopo un'interruzione di corrente […] [ticket-supporto-003]

D: E per il secondo caso?
  1. manuale-lumadesk-014  score=0.388  Se il display resta spento, prova un'altra presa…
  2. guida-smart-working-006  score=0.380  Da casa valgono le stesse regole dell'ufficio…
  … (a schermo: quattro chunk)
→ Non lo trovo nei documenti.
```

Il retrieval ha cercato letteralmente "**secondo caso**": i chunk recuperati non c'entrano niente, e la risposta — correttamente — si rifiuta.



Single-turn per costruzione

`query` → **recupera()** → **rispondi()** → risposta

~~storia della conversazione~~

non entra da nessuna parte

> Nessuno l'ha scritta male: dentro `recupera()` entra solo la domanda così com'è, e «E per il secondo caso?» non contiene **nessuna** delle parole che servono a trovare il chunk giusto.



La prossima lezione

`«Non lo trovo nei documenti.»` → **Modulo 3 · Lezione 3 — Lab: RAG Conversazionale**

- memoria della conversazione
- follow-up che funzionano
- sessioni persistenti

Si parte esattamente dal fallimento appena visto.



Grazie

prossima lezione: Modulo 3 · Lezione 3 — Lab: RAG Conversazionale

